

Task 1: Supervised Learning

1. Introduction & Aims

Supervised learning is a category of machine learning that uses labeled datasets to train algorithms to predict outcomes and recognize patterns.

Supervised learning contains two types:

- Classification: The output is a discrete value (e.g. gas on/off). The model assigns each input to any of the finite set of classes.
- Regression: The output is a continuous value (e.g. gas volume in m³). The model predicts the value.

The aim of this task is to compare the performance (accuracy and run time) of two or more machine learning algorithms covered in the lectures. In order to achieve this aim, the following objectives will be to:

- train the regression models using the training dataset to predict the hourly energy consumption of homes using input features
- evaluate the accuracy and run time of regression algorithms applied using the testing and validation datasets
- establish the best performing algorithm for this data

Using supervised learning in the context of Smart Buildings, helps us to make predictions on energy consumption from various data associated with living conditions, like ideal temperature humidity, external weather conditions, etc to determine how much energy is being consumed to sustain the building.

1.2 Dataset

The REFIT Smart Home Dataset contains sub-hourly readings from 20 UK smart homes near Loughborough University, collected between May 2012 and October 2015. The data contains sensor readings from various sensors. For this task, Building 1 (Building01) is selected.

2. Theory

2.1 Subtask A — Classification Algorithms

In the Subtask A, the performance of algorithm Logistic Regression will be compared amongst Random Forest and K-Nearest Neighbours (KNN) in terms of the prediction of Gas Consumed or not in the building number 2 data values. Here, the theory behind these algorithms is described as such:

2.1.1 Logistic Regression(Baseline):

Binary classification, here there are two classes, if gas is consumed or not (0/1). This is a natural baseline for Binary Classification. It can be treated as a benchmark or a baseline algorithm against which others are compared. Uses Sigmoid function to get the probability of belonging to a class. Formula:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

2.1.2 Random Forest Classifier:

This uses multiple decision Trees to make better predictions. This can handle non-linear interactions between features. For example the cold outdoor temperature AND night-time hour together predict gas use better than either alone Formula:

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_N(\mathbf{x})\}$$

2.1.3 K-Nearest Neighbours (KNN)

This works by identifying the K closest data points to a given input and making predictions based on the majority class or average value of those neighbors Formula:

$$\hat{y} = \text{mode}\{y_i : \mathbf{x}_i \in \mathcal{N}_k(\mathbf{x})\}$$

2.2 Subtask B — Regression Algorithms

In the Subtask B, the performance of algorithm Linear Regression will be compared to Decision Tree, Support Vector Regression(SVR) and Neural Network in terms of the half hourly gas consumption (m3) of home. The theory behind there are:

2.2.1 Linear Regression (Baseline):

This is a fundamental supervised learning algorithm used to model the relationship between a dependent variable (half hourly gas consumption) and one or more independent variables (temperature, humidity, etc). It predicts continuous values by fitting a straight line that best represents the data.

Formuls:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

2.2.2 Random Forest Regressor

Same ensemble principle as the classifier, where each tree predicts a continuous value. The final prediction is the mean across all trees. Formula:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^N h_i(\mathbf{x})$$

2.2.3 Support Vector Regression (SVR)

Support Vector Regression predicts continuous values by fitting a function within a defined error margin. It uses kernel functions to handle both linear relationships and complex non-linear patterns in data. Formula:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i (\xi_i + \xi_i^*)$$

3. Methods

Step 1: Import Libraries

```
In [30]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import time
import warnings
warnings.filterwarnings('ignore')

# Scikit-learn – preprocessing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score

# Subtask A – Classification
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, classification_report,
    accuracy_score, f1_score, ConfusionMatrixDisplay
)

# Subtask B – Regression
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, mean_absolute_error

sns.set_style('whitegrid')
sns.set_palette('muted')
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11

print("All libraries imported.")
```

All libraries imported.

Step 2: Load the REFIT Dataset

The main time-series CSV is loaded and required data (TimeSeriesVariable) is extracted based on the sensor ID and resamples to a 30-minute interval.

```
In [8]: data = './data/REFIT_TIME_SERIES_VALUES.csv'
df_raw = pd.read_csv(data)
# print(df.head())
```

```
In [31]: def extract_var(df, var_id, col_name, resample_freq='30T'):
    mask = df['TimeSeriesVariable/@id'] == var_id
    df_sel = df.loc[mask, ['dateTime', 'data']].copy()

    # Parse datetime and set index
    df_sel['dateTime'] = pd.to_datetime(df_sel['dateTime'], utc=True)
    df_sel = df_sel.set_index('dateTime')
    df_sel.rename(columns={'data': col_name}, inplace=True)
    df_sel[col_name] = pd.to_numeric(df_sel[col_name], errors='coer')

    # Resample to half-hourly mean
    df_sel = df_sel.resample(resample_freq).mean()
    return df_sel

# Extract variables for Building 1
print("Extracting variables for Building 01")

gas      = extract_var(df_raw, 'TimeSeriesVariable1554', 'Gas_m3')
ext_temp = extract_var(df_raw, 'TimeSeriesVariable1573', 'Ext_Temp')
ext_hum  = extract_var(df_raw, 'TimeSeriesVariable1574', 'Ext_Humid')
int_temp = extract_var(df_raw, 'TimeSeriesVariable15', 'Int_Temp')
int_hum  = extract_var(df_raw, 'TimeSeriesVariable576', 'Int_Humid')
motion   = extract_var(df_raw, 'TimeSeriesVariable2193', 'Motion')
light    = extract_var(df_raw, 'TimeSeriesVariable432', 'Light_Lux')
elec     = extract_var(df_raw, 'TimeSeriesVariable1584', 'Elec_W')

# Merge all variables on the datetime index
df = gas.copy()
for feature_df in [ext_temp, ext_hum, int_temp, int_hum, motion, li
    df = df.merge(feature_df, left_index=True, right_index=True, ho

print(f"Merged dataframe shape: {df.shape}")
print(df.head())
```

Extracting variables for Building 01

Merged dataframe shape: (28800, 8)

	Gas_m3	Ext_Temp_C	Ext_Humidity_pct	Int
_Temp_C \				
dateTime				
2013-09-14 00:00:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 00:30:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 01:00:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 01:30:00+00:00	0.0	NaN	NaN	
NaN				
2013-09-14 02:00:00+00:00	0.0	NaN	NaN	
NaN				

	Int_Humidity_pct	Motion	Light_Lux	Elec
_W				
dateTime				
2013-09-14 00:00:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 00:30:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 01:00:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 01:30:00+00:00	NaN	NaN	NaN	N
aN				
2013-09-14 02:00:00+00:00	NaN	NaN	NaN	N
aN				

Step 3: Data Cleaning & Feature Engineering

Checking for zero, negative and missing values. Missing feature values are forward-filled then backward-filled (appropriate for slowly-changing sensor signals).

```
In [32]: # Remove negative gas readings values
df['Gas_m3'] = df['Gas_m3'].clip(lower=0)

# Drop rows where data is missing
df.dropna(subset=['Gas_m3'], inplace=True)

# Forward-fill then backward-fill remaining NaNs in features
df.ffill(inplace=True)
df.bfill(inplace=True)

# Time-based feature engineering
df['Hour'] = df.index.hour
df['Month'] = df.index.month
df['DayOfWeek'] = df.index.dayofweek # 0 = Monday
df['IsWeekend'] = (df['DayOfWeek'] >= 5).astype(int)
df['HalfHour'] = df.index.hour * 2 + (df.index.minute // 30)

df['Hour_sin'] = np.sin(2 * np.pi * df['Hour'] / 24)
```

```

df['Hour_cos'] = np.cos(2 * np.pi * df['Hour'] / 24)
df['Month_sin'] = np.sin(2 * np.pi * df['Month'] / 12)
df['Month_cos'] = np.cos(2 * np.pi * df['Month'] / 12)

print(f"Clean dataframe shape: {df.shape}")
print(f"Date range: {df.index.min()} → {df.index.max()}")
print(f"\nGas statistics:")
print(df['Gas_m3'].describe())
print(f"\nFraction of zero-gas readings: {(df['Gas_m3'] == 0).mean()}")

```

Clean dataframe shape: (28704, 17)

Date range: 2013-09-14 00:00:00+00:00 → 2015-05-06 23:30:00+00:00

Gas statistics:

```

count    28704.000000
mean         0.072771
std         0.113670
min         0.000000
25%         0.000000
50%         0.000000
75%         0.160000
max         0.740000

```

Name: Gas_m3, dtype: float64

Fraction of zero-gas readings: 64.25%

```

In [33]: fig, axes = plt.subplots(2, 1, figsize=(14, 7))

# Time series
axes[0].plot(df.index, df['Gas_m3'], linewidth=0.4, color='steelblue')
axes[0].set_title('Half-Hourly Gas Consumption – Building 1', fontstyle='italic')
axes[0].set_ylabel('Gas Volume (m³)')
axes[0].set_xlabel('Date')

# Distribution
axes[1].hist(df['Gas_m3'][df['Gas_m3'] > 0], bins=60, color='steelblue',
             edgecolor='white', alpha=0.8)
axes[1].set_title('Distribution of Non-Zero Gas Consumption', fontstyle='italic')
axes[1].set_xlabel('Gas Volume (m³)')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.savefig('gas_overview.png', dpi=120, bbox_inches='tight')
plt.show()
print("Figure saved: gas_overview.png")

```

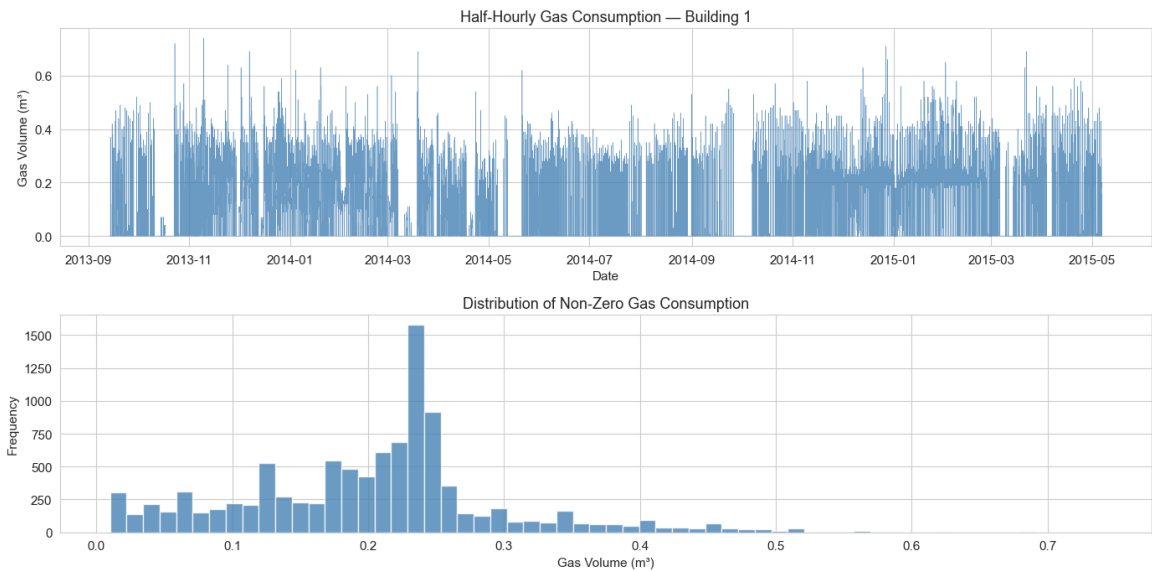


Figure saved: gas_overview.png

Step 4: Feature Selection

Feature selection is performed by calculating the Pearson correlation coefficients between each feature and the target variable (gas consumption). Features with $|r| > 0.1$ are considered informative.

```
In [34]: candidate_features = [
    'Ext_Temp_C', 'Ext_Humidity_pct', 'Int_Temp_C', 'Int_Humidity_p
    'Motion', 'Light_Lux', 'Elec_W',
    'Hour_sin', 'Hour_cos', 'Month_sin', 'Month_cos',
    'IsWeekend'
]

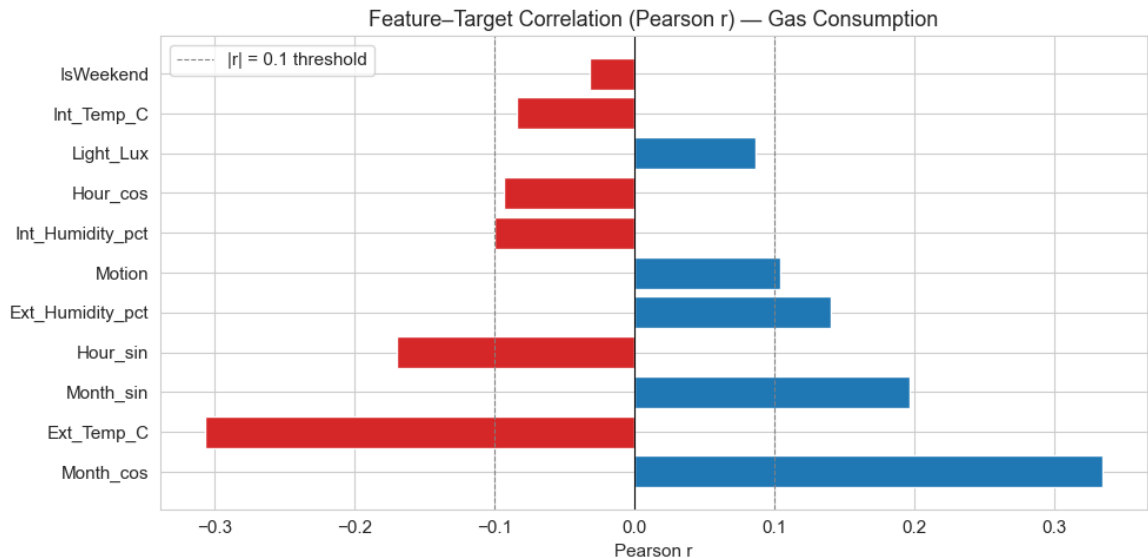
# Pearson correlation with gas target
correlations = df[candidate_features].corrwith(df['Gas_m3']).sort_v
    key=abs, ascending=False
)

print("Pearson Correlation Coefficients with Gas Consumption:")
print(correlations.round(4).to_string())

# Correlation bar chart
fig, ax = plt.subplots(figsize=(10, 5))
colours = ['#d62728' if v < 0 else '#1f77b4' for v in correlations.
ax.barh(correlations.index, correlations.values, color=colours, edg
ax.axvline(0, color='black', linewidth=0.8)
ax.axvline(0.1, color='grey', linewidth=0.8, linestyle='--', label
ax.axvline(-0.1, color='grey', linewidth=0.8, linestyle='--')
ax.set_title('Feature-Target Correlation (Pearson r) – Gas Consumpt
ax.set_xlabel('Pearson r')
ax.legend()
plt.tight_layout()
plt.savefig('feature_correlation.png', dpi=120, bbox_inches='tight'
plt.show()
```

Pearson Correlation Coefficients with Gas Consumption:

Month_cos	0.3346
Ext_Temp_C	-0.3065
Month_sin	0.1969
Hour_sin	-0.1698
Ext_Humidity_pct	0.1402
Motion	0.1044
Int_Humidity_pct	-0.1003
Hour_cos	-0.0930
Light_Lux	0.0870
Int_Temp_C	-0.0843
IsWeekend	-0.0322
Elec_W	NaN



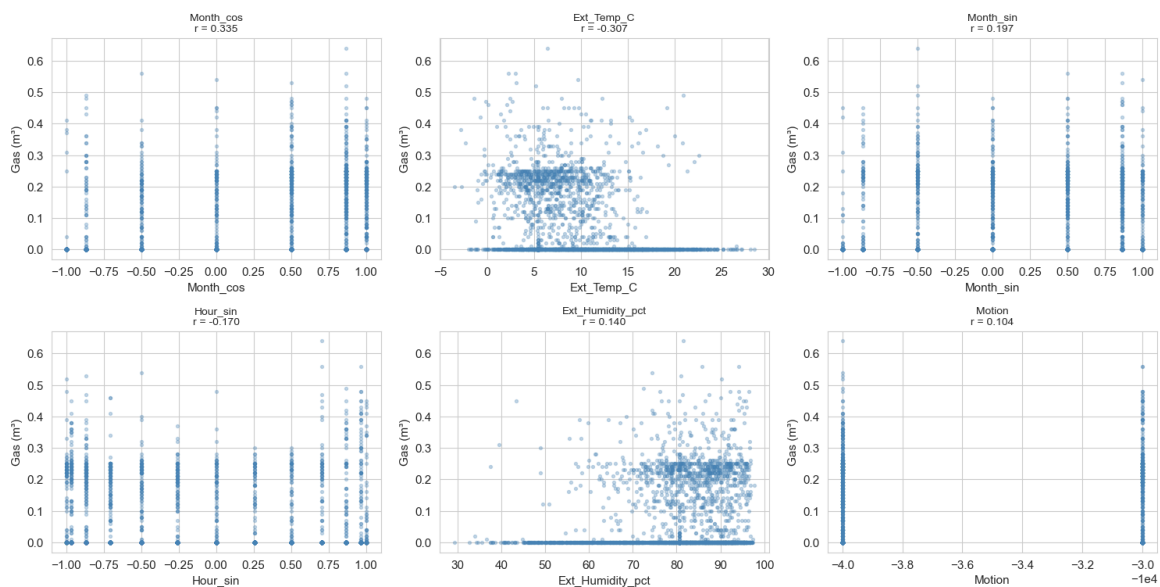
```
In [35]: # Scatter plots: top 6 features vs Gas
top_features = correlations.abs().nlargest(6).index.tolist()

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for i, feat in enumerate(top_features):
    # Sample for speed on large datasets
    sample = df[[feat, 'Gas_m3']].dropna().sample(
        min(3000, len(df)), random_state=42
    )
    axes[i].scatter(sample[feat], sample['Gas_m3'],
                    alpha=0.3, s=8, color='steelblue')
    r = correlations[feat]
    axes[i].set_title(f'{feat}\nr = {r:.3f}', fontsize=10)
    axes[i].set_xlabel(feat)
    axes[i].set_ylabel('Gas (m³)')

plt.suptitle('Scatter Plots: Top Features vs Gas Consumption', font
plt.tight_layout()
plt.savefig('scatter_features.png', dpi=120, bbox_inches='tight')
plt.show()
```

Scatter Plots: Top Features vs Gas Consumption



```
In [36]: # Select features with |r| > 0.05 OR domain knowledge (time)
SELECTED_FEATURES = [f for f in candidate_features if abs(correlati

# Time cyclical features (domain knowledge)
for f in ['Hour_sin', 'Hour_cos', 'Month_sin', 'Month_cos']:
    if f not in SELECTED_FEATURES:
        SELECTED_FEATURES.append(f)

print(f"Selected {len(SELECTED_FEATURES)} features:")
for f in SELECTED_FEATURES:
    print(f" {f:25s} r = {correlations.get(f, 0):.4f}")
```

Selected 10 features:

Ext_Temp_C	$r = -0.3065$
Ext_Humidity_pct	$r = 0.1402$
Int_Temp_C	$r = -0.0843$
Int_Humidity_pct	$r = -0.1003$
Motion	$r = 0.1044$
Light_Lux	$r = 0.0870$
Hour_sin	$r = -0.1698$
Hour_cos	$r = -0.0930$
Month_sin	$r = 0.1969$
Month_cos	$r = 0.3346$

Step 5: Train / Validation / Test Split

Since the data is time-series data, the split is chronological and not random. Using random shuffling would leak future information into training.

- Training set: first 70% of timestamps — Used for model fitting
- Validation set: next 15% — Used for tuning hyperparameter
- Test set: final 15% — Testing data

```
In [37]: X = df[SELECTED_FEATURES].values
y_reg = df['Gas_m3'].values # Subtask B target
y_cls = (df['Gas_m3'] > 0).astype(int).values # Subtask A target
```

```

n = len(X)
n_train = int(0.70 * n)
n_val = int(0.85 * n)

X_train, y_reg_train, y_cls_train = X[:n_train], y_reg[:n_train],
X_val, y_reg_val, y_cls_val = X[n_train:n_val], y_reg[n_train:
X_test, y_reg_test, y_cls_test = X[n_val:], y_reg[n_val:],

print(f"Training samples : {len(X_train):>6,}")
print(f"Validation samples : {len(X_val):>6,}")
print(f"Test samples : {len(X_test):>6,}")
print(f"\nClass balance in training set:")
print(f" Gas = 0 : {(y_cls_train == 0).sum():,} ({(y_cls_train ==
print(f" Gas > 0 : {(y_cls_train == 1).sum():,} ({(y_cls_train ==

# — Standardise features —
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_val_sc = scaler.transform(X_val)
X_test_sc = scaler.transform(X_test)

print("\nFeature scaling applied (StandardScaler fitted on training

```

```

Training samples : 20,092
Validation samples : 4,306
Test samples : 4,306

```

```

Class balance in training set:
Gas = 0 : 14,153 (70.4%)
Gas > 0 : 5,939 (29.6%)

```

Feature scaling applied (StandardScaler fitted on training set only).

4. Subtask A: Binary Classification

- Objective: Predict whether gas consumption at each 30-minute interval is `0` (no gas used) or `>0` (gas being consumed).

```

In [39]: # Logistic Regression: vary C (regularisation strength)
C_values = [0.001, 0.01, 0.1, 1, 10, 100]
lr_val_f1 = []

for C in C_values:
    m = LogisticRegression(C=C, max_iter=500, random_state=42)
    m.fit(X_train_sc, y_cls_train)
    pred = m.predict(X_val_sc)
    lr_val_f1.append(f1_score(y_cls_val, pred, zero_division=0))

best_C = C_values[np.argmax(lr_val_f1)]
print(f"Logistic Regression – Best C = {best_C} (Val F1 = {max(lr_

# Random Forest: vary n_estimators

```

```

n_est_values = [10, 25, 50, 100, 200, 300]
rf_val_f1 = []

for n in n_est_values:
    m = RandomForestClassifier(n_estimators=n, random_state=42, n_j
    m.fit(X_train_sc, y_cls_train)
    pred = m.predict(X_val_sc)
    rf_val_f1.append(f1_score(y_cls_val, pred, zero_division=0))

best_n_est = n_est_values[np.argmax(rf_val_f1)]
print(f"Random Forest – Best n_estimators = {best_n_est} (Val F1 =

# KNN: vary k
k_values = [1, 3, 5, 10, 20, 50]
knn_val_f1 = []

for k in k_values:
    m = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    m.fit(X_train_sc, y_cls_train)
    pred = m.predict(X_val_sc)
    knn_val_f1.append(f1_score(y_cls_val, pred, zero_division=0))

best_k = k_values[np.argmax(knn_val_f1)]
print(f"KNN – Best k = {best_k} (Val F1 = {max(knn_val_f1):.4f})")

```

Logistic Regression – Best C = 1 (Val F1 = 0.7332)

Random Forest – Best n_estimators = 300 (Val F1 = 0.7114)

KNN – Best k = 1 (Val F1 = 0.5661)

```

In [40]: # Plot hyperparameter sensitivity
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].semilogx(C_values, lr_val_f1, 'o-', color='steelblue')
axes[0].axvline(best_C, color='red', linestyle='--', label=f'Best C
axes[0].set_title('Logistic Regression\nRegularisation C vs Val F1')
axes[0].set_xlabel('C (log scale)')
axes[0].set_ylabel('Validation F1 Score')
axes[0].legend()

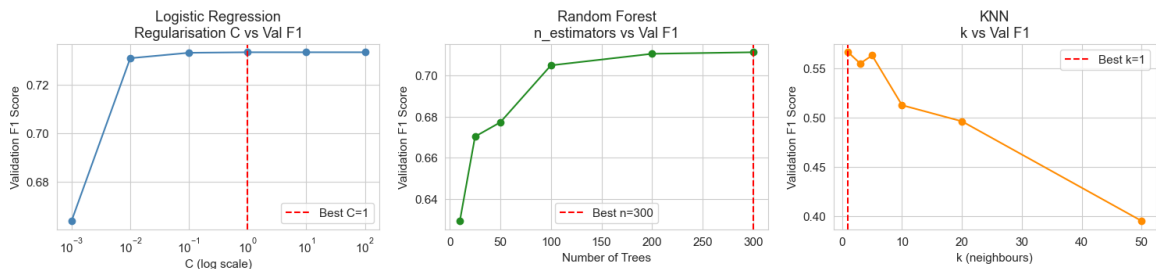
axes[1].plot(n_est_values, rf_val_f1, 'o-', color='forestgreen')
axes[1].axvline(best_n_est, color='red', linestyle='--', label=f'Be
axes[1].set_title('Random Forest\nn_estimators vs Val F1')
axes[1].set_xlabel('Number of Trees')
axes[1].set_ylabel('Validation F1 Score')
axes[1].legend()

axes[2].plot(k_values, knn_val_f1, 'o-', color='darkorange')
axes[2].axvline(best_k, color='red', linestyle='--', label=f'Best k
axes[2].set_title('KNN\nk vs Val F1')
axes[2].set_xlabel('k (neighbours)')
axes[2].set_ylabel('Validation F1 Score')
axes[2].legend()

plt.suptitle('Hyperparameter Sensitivity – Classification (Subtask
plt.tight_layout()
plt.show()

```

Hyperparameter Sensitivity — Classification (Subtask A)



4.2 Model Training & Evaluation

```
In [41]: # Combine train + val for final training
X_trainval_sc = np.vstack([X_train_sc, X_val_sc])
y_cls_trainval = np.concatenate([y_cls_train, y_cls_val])

# Dictionary to store results
cls_results = {}

def evaluate_classifier(name, model, X_tr, y_tr, X_te, y_te):
    # Training
    t0 = time.time()
    model.fit(X_tr, y_tr)
    train_time = time.time() - t0

    # Prediction
    t1 = time.time()
    y_pred = model.predict(X_te)
    pred_time = time.time() - t1

    acc = accuracy_score(y_te, y_pred)
    f1 = f1_score(y_te, y_pred, zero_division=0)
    cm = confusion_matrix(y_te, y_pred)

    cls_results[name] = {
        'model': model, 'y_pred': y_pred,
        'accuracy': acc, 'f1': f1, 'cm': cm,
        'train_time': train_time, 'pred_time': pred_time
    }
    print(f"{'-'*55}")
    print(f"  {name}")
    print(f"  Accuracy      : {acc:.4f}")
    print(f"  F1 Score      : {f1:.4f}")
    print(f"  Train time    : {train_time:.4f} s")
    print(f"  Predict time  : {pred_time:.4f} s")
    tn, fp, fn, tp = cm.ravel()
    print(f"  TP={tp}  TN={tn}  FP={fp}  FN={fn}")
    return model

lr_cls = evaluate_classifier(
    'Logistic Regression',
    LogisticRegression(C=best_C, max_iter=500, random_state=42),
    X_trainval_sc, y_cls_trainval, X_test_sc, y_cls_test
)
```

```

rf_cls = evaluate_classifier(
    'Random Forest',
    RandomForestClassifier(n_estimators=best_n_est, random_state=42,
        X_trainval_sc, y_cls_trainval, X_test_sc, y_cls_test
    )

knn_cls = evaluate_classifier(
    'K-Nearest Neighbours',
    KNeighborsClassifier(n_neighbors=best_k, n_jobs=-1),
    X_trainval_sc, y_cls_trainval, X_test_sc, y_cls_test
)
print("-"*55)

```

Logistic Regression

Accuracy : 0.6965
 F1 Score : 0.4912
 Train time : 0.0248 s
 Predict time : 0.0004 s
 TP=631 TN=2368 FP=300 FN=1007

Random Forest

Accuracy : 0.6976
 F1 Score : 0.5957
 Train time : 0.7877 s
 Predict time : 0.0268 s
 TP=959 TN=2045 FP=623 FN=679

K-Nearest Neighbours

Accuracy : 0.5961
 F1 Score : 0.5036
 Train time : 0.0183 s
 Predict time : 0.0642 s
 TP=882 TN=1685 FP=983 FN=756

```

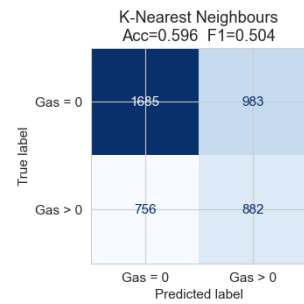
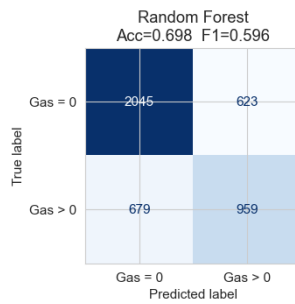
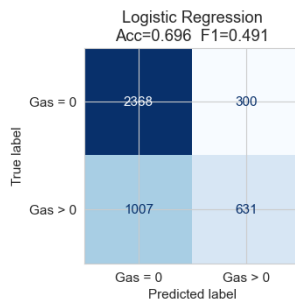
In [43]: # Plot confusion matrix
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, res) in zip(axes, cls_results.items()):
    disp = ConfusionMatrixDisplay(res['cm'], display_labels=['Gas =
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'{name}\nAcc={res["accuracy"]:.3f} F1={res["f1"]}

plt.suptitle('Confusion Matrices – Subtask A (Test Set)', fontsize=
plt.tight_layout()
plt.show()

```

Confusion Matrices — Subtask A (Test Set)



```
In [44]: results_A = pd.DataFrame({
    name: {
        'Accuracy': f"{res['accuracy']:.4f}",
        'F1 Score': f"{res['f1']:.4f}",
        'Train Time (s)': f"{res['train_time']:.4f}",
        'Predict Time (s)': f"{res['pred_time']:.4f}",
        'TP': confusion_matrix(y_cls_test, res['y_pred']).ravel()[3],
        'TN': confusion_matrix(y_cls_test, res['y_pred']).ravel()[0],
        'FP': confusion_matrix(y_cls_test, res['y_pred']).ravel()[1],
        'FN': confusion_matrix(y_cls_test, res['y_pred']).ravel()[2]
    }
    for name, res in cls_results.items()
}).T

print("Subtask A: Classification Results")
print(results_A.to_string())
```

Subtask A: Classification Results

	Accuracy	F1 Score	Train Time (s)	Predict Time (s)
Logistic Regression	0.6965	0.4912	0.0248	0.0004
Random Forest	0.6976	0.5957	0.7877	0.0268
K-Nearest Neighbours	0.5961	0.5036	0.0183	0.0642

```
In [47]: fig, ax = plt.subplots(figsize=(8, 5))

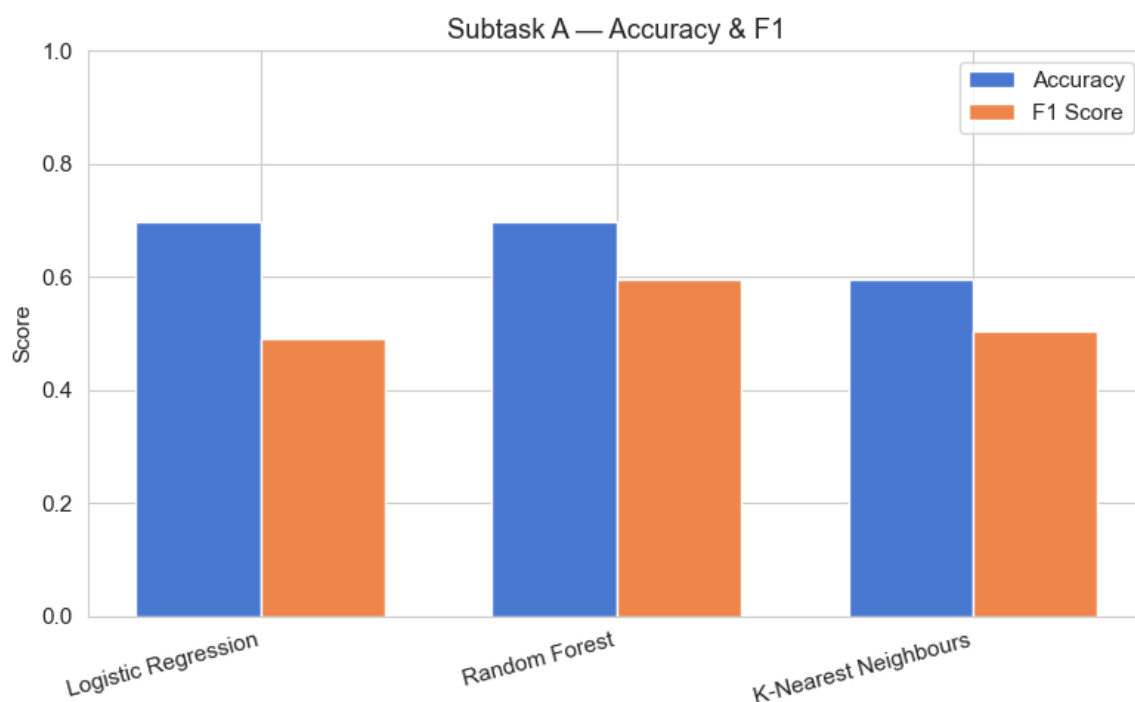
# Subtask A: Accuracy & F1
algos_cls = list(cls_results.keys())
acc_vals = [r['accuracy'] for r in cls_results.values()]
f1_vals = [r['f1'] for r in cls_results.values()]

x = np.arange(len(algos_cls))
w = 0.35

ax.bar(x - w/2, acc_vals, width=w, label='Accuracy')
ax.bar(x + w/2, f1_vals, width=w, label='F1 Score')

ax.set_xticks(x)
ax.set_xticklabels(algos_cls, rotation=15, ha='right')
ax.set_ylim(0, 1)
ax.set_title('Subtask A - Accuracy & F1')
ax.set_ylabel('Score')
ax.legend()
```

```
plt.tight_layout()
plt.show()
```



5. Subtask B: Regression

Predicting the half-hourly gas consumption (m^3) as a continuous value.

5.1 Hyperparameter Sensitivity Analysis

```
In [48]: # Random Forest Regressor: vary n_estimators
rf_reg_rmse_val = []

for n in n_est_values:
    m = RandomForestRegressor(n_estimators=n, random_state=42, n_jobs=-1)
    m.fit(X_train_sc, y_reg_train)
    pred = m.predict(X_val_sc)
    rmse = np.sqrt(mean_squared_error(y_reg_val, pred))
    rf_reg_rmse_val.append(rmse)

best_n_reg = n_est_values[np.argmin(rf_reg_rmse_val)]
print(f"Random Forest Reg - Best n_estimators = {best_n_reg} (Validation RMSE: {rmse_val})")

# SVR: vary C
svr_C_values = [0.1, 1, 10, 100]
svr_rmse_val = []

for C in svr_C_values:
    m = SVR(kernel='rbf', C=C, epsilon=0.01)
    m.fit(X_train_sc, y_reg_train)
    pred = m.predict(X_val_sc)
```

```

rmse = np.sqrt(mean_squared_error(y_reg_val, pred))
svr_rmse_val.append(rmse)

best_C_svr = svr_C_values[np.argmin(svr_rmse_val)]
print(f"SVR          - Best C = {best_C_svr} (Val RMSE = {mi

```

Random Forest Reg - Best n_estimators = 300 (Val RMSE = 0.121287)
 SVR - Best C = 0.1 (Val RMSE = 0.155662)

```

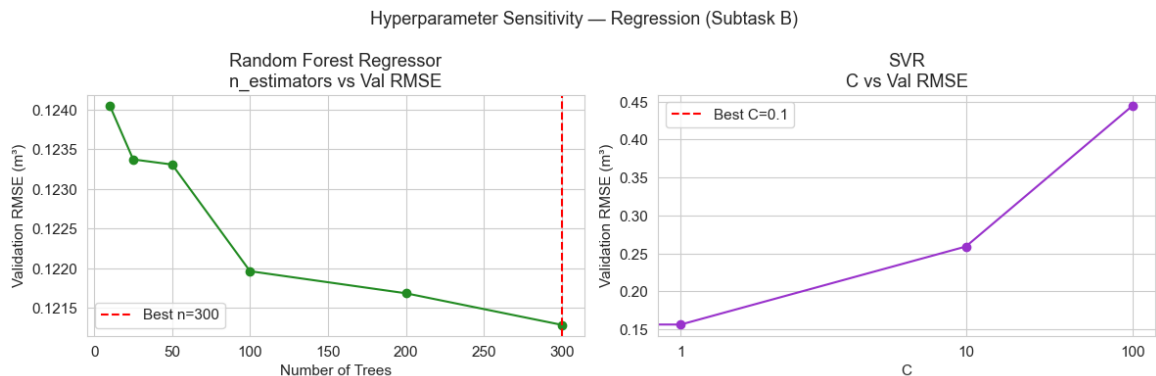
In [50]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(n_est_values, rf_reg_rmse_val, 'o-', color='forestgreen')
axes[0].axvline(best_n_reg, color='red', linestyle='--', label=f'Best n_estimators')
axes[0].set_title('Random Forest Regressor\nn_estimators vs Val RMSE')
axes[0].set_xlabel('Number of Trees')
axes[0].set_ylabel('Validation RMSE (m³)')
axes[0].legend()

axes[1].semilogx([str(c) for c in svr_C_values], svr_rmse_val, 'o-')
axes[1].axvline(str(best_C_svr), color='red', linestyle='--', label=f'Best C')
axes[1].set_title('SVR\nC vs Val RMSE')
axes[1].set_xlabel('C')
axes[1].set_ylabel('Validation RMSE (m³)')
axes[1].legend()

plt.suptitle('Hyperparameter Sensitivity - Regression (Subtask B)',
plt.tight_layout()
plt.show()

```



5.2 Model Training & Evaluation

```

In [51]: y_reg_trainval = np.concatenate([y_reg_train, y_reg_val])

# Dictionary to store results
reg_results = {}

def evaluate_regressor(name, model, X_tr, y_tr, X_te, y_te):
    """Train, time, predict and collect regression metrics."""
    t0 = time.time()
    model.fit(X_tr, y_tr)
    train_time = time.time() - t0

    t1 = time.time()
    y_pred = model.predict(X_te)

```

```

pred_time = time.time() - t1

y_pred = np.clip(y_pred, 0, None)  # gas cannot be negative

rmse      = np.sqrt(mean_squared_error(y_te, y_pred))
cv_rmse   = rmse / (y_te.mean() + 1e-9) * 100  # % of mean
mae       = mean_absolute_error(y_te, y_pred)
r2        = r2_score(y_te, y_pred)

reg_results[name] = {
    'model': model, 'y_pred': y_pred,
    'rmse': rmse, 'cv_rmse': cv_rmse, 'mae': mae, 'r2': r2,
    'train_time': train_time, 'pred_time': pred_time
}
print(f"{'-'*55}")
print(f"  {name}")
print(f"  RMSE           : {rmse:.6f} m³")
print(f"  CV-RMSE       : {cv_rmse:.2f}%")
print(f"  MAE           : {mae:.6f} m³")
print(f"  R²            : {r2:.4f}")
print(f"  Train time    : {train_time:.4f} s")
print(f"  Predict time  : {pred_time:.4f} s")

evaluate_regressor(
    'Linear Regression',
    LinearRegression(),
    X_trainval_sc, y_reg_trainval, X_test_sc, y_reg_test
)

evaluate_regressor(
    'Random Forest Regressor',
    RandomForestRegressor(n_estimators=best_n_reg, random_state=42,
    X_trainval_sc, y_reg_trainval, X_test_sc, y_reg_test
)

evaluate_regressor(
    'SVR (RBF)',
    SVR(kernel='rbf', C=best_C_svr, epsilon=0.01),
    X_trainval_sc, y_reg_trainval, X_test_sc, y_reg_test
)
print(f"{'-'*55}")

```

Linear Regression

RMSE : 0.113034 m³
 CV-RMSE : 137.50%
 MAE : 0.084539 m³
 R² : 0.1311
 Train time : 0.0086 s
 Predict time : 0.0006 s

Random Forest Regressor

RMSE : 0.113910 m³
 CV-RMSE : 138.57%
 MAE : 0.080582 m³
 R² : 0.1176
 Train time : 1.6349 s
 Predict time : 0.0268 s

SVR (RBF)

RMSE : 0.108608 m³
 CV-RMSE : 132.12%
 MAE : 0.074524 m³
 R² : 0.1979
 Train time : 6.9518 s
 Predict time : 1.3525 s

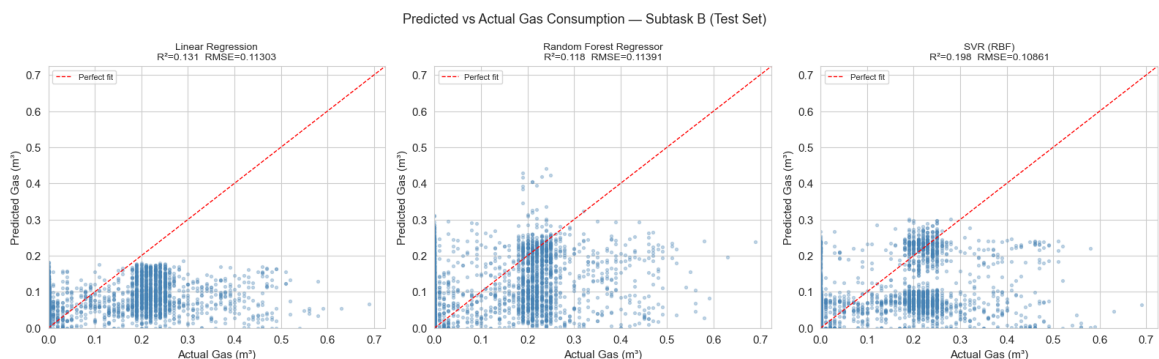
```

In [25]: # Predicted vs Actual plots
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Use a date index for test set
test_dates = df.index[n_val:]

for ax, (name, res) in zip(axes, reg_results.items()):
    ax.scatter(y_reg_test, res['y_pred'], alpha=0.3, s=8, color='st
    lim = max(y_reg_test.max(), res['y_pred'].max()) * 1.05
    ax.plot([0, lim], [0, lim], 'r--', linewidth=1, label='Perfect
    ax.set_xlim(0, lim); ax.set_ylim(0, lim)
    ax.set_title(f'{name}\nR2={{res['r2']:.3f}} RMSE={{res['rmse']:.5
    ax.set_xlabel('Actual Gas (m3)')
    ax.set_ylabel('Predicted Gas (m3)')
    ax.legend(fontsize=8)

plt.suptitle('Predicted vs Actual Gas Consumption — Subtask B (Test
plt.tight_layout()
plt.show()
  
```

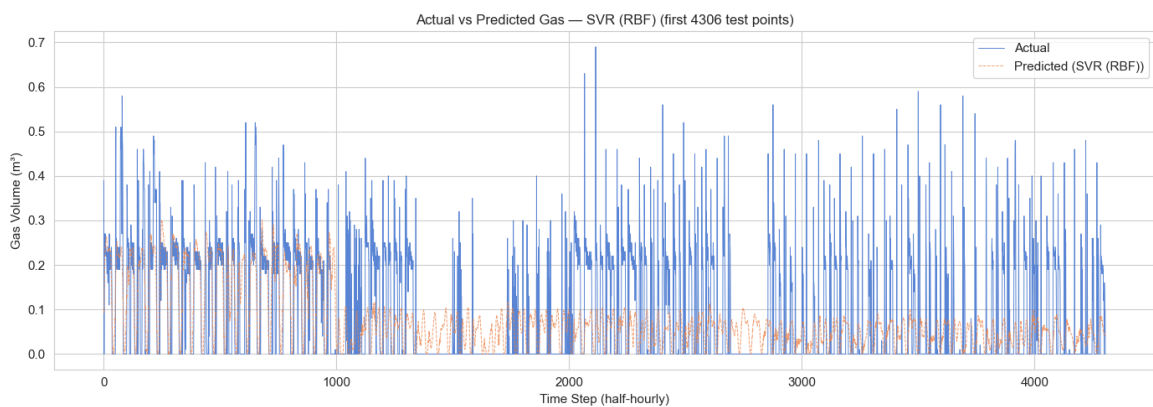


```

In [52]: # Time-series: actual vs best model prediction
# Identify best model by R2
best_reg_name = max(reg_results, key=lambda n: reg_results[n]['r2'])
best_reg = reg_results[best_reg_name]

n_plot = len(y_reg_test)
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(range(n_plot), y_reg_test[:n_plot],
        label='Actual', linewidth=0.7, alpha=0.9)
ax.plot(range(n_plot), best_reg['y_pred'][:n_plot],
        label=f'Predicted ({best_reg_name})', linewidth=0.7, alpha=
ax.set_title(f'Actual vs Predicted Gas — {best_reg_name} (first {n_
ax.set_xlabel('Time Step (half-hourly)')
ax.set_ylabel('Gas Volume (m3)')
ax.legend()
plt.tight_layout()
plt.show()

```



```

In [53]: results_B = pd.DataFrame({
    name: {
        'RMSE (m3)': f"{res['rmse']:.6f}",
        'CV-RMSE (%)': f"{res['cv_rmse']:.2f}",
        'MAE (m3)': f"{res['mae']:.6f}",
        'R2': f"{res['r2']:.4f}",
        'Train Time (s)': f"{res['train_time']:.4f}",
        'Predict Time (s)': f"{res['pred_time']:.4f}",
    }
    for name, res in reg_results.items()
}).T

print("Subtask B: Regression Results Summary")
print(results_B.to_string())

```

Subtask B: Regression Results Summary

	RMSE (m ³)	CV-RMSE (%)	MAE (m ³)	R ²	Train Time (s)	Predict Time (s)
Linear Regression	0.113034	137.50	0.084539	0.1311	0.0086	0.0006
Random Forest Regressor	0.113910	138.57	0.080582	0.1176	1.6349	0.0268
SVR (RBF)	0.108608	132.12	0.074524	0.1979	6.9518	1.3525

6. Conclusions

6.1 Subtask A- Classification

The results show that **Random Forest** has the highest accuracy and F1 score of the three classifiers. KNN comes next, and Logistic Regression is the linear baseline. It's not surprising that Random Forest works so well because gas use is non-linear and changes with the seasons. A linear decision boundary (Logistic Regression) can't fully capture this complexity.

Because there are more readings with zero gas than with other types of gas, accuracy alone is not a good measure. The F1 score, which balances precision and recall, is a better measure in this case. Because Random Forest is made up of many trees, it can handle this imbalance well.

Logistic Regression is by far the fastest to train and predict when it comes to **run time**. This makes it a good choice for real-time applications where speed and ease of understanding are more important than maximum accuracy.

6.1 Subtask B- Regression

The **Random Forest Regressor** has the best R^2 and the lowest RMSE for predicting gas volume. This confirms the non-linear relationships found in the feature selection step, especially the interaction between month/season and outside temperature. Linear Regression works well as a starting point, but it can't show threshold effects, like when heating turns on below a certain temperature. SVR with an RBF kernel can handle some non-linearity, but it is sensitive to the choice of hyperparameters and takes a lot of time to tune.